

End-to-End CI/CD Pipeline Project Report

Automated Build, Containerization, and Deployment on AWS ECS using GitHub Actions & ECR

PROJECT: train-reservation-ticket

CONTAINER REGISTRY: Amazon ECR (train-book)

ORCHESTRATION: Amazon ECS (Fargate / EC2)

CI/CD ENGINE: GitHub Actions Workflow

1. Project Overview & Architecture

This project establishes a fully automated Continuous Integration and Continuous Deployment (CI/CD) delivery pipeline for a Java/Spring Boot web application (train-reservation-ticket). Whenever new code is pushed to the main branch on GitHub, the pipeline automatically compiles the source code, packages the Docker image, pushes it to Amazon ECR, and triggers a rolling update on Amazon ECS.

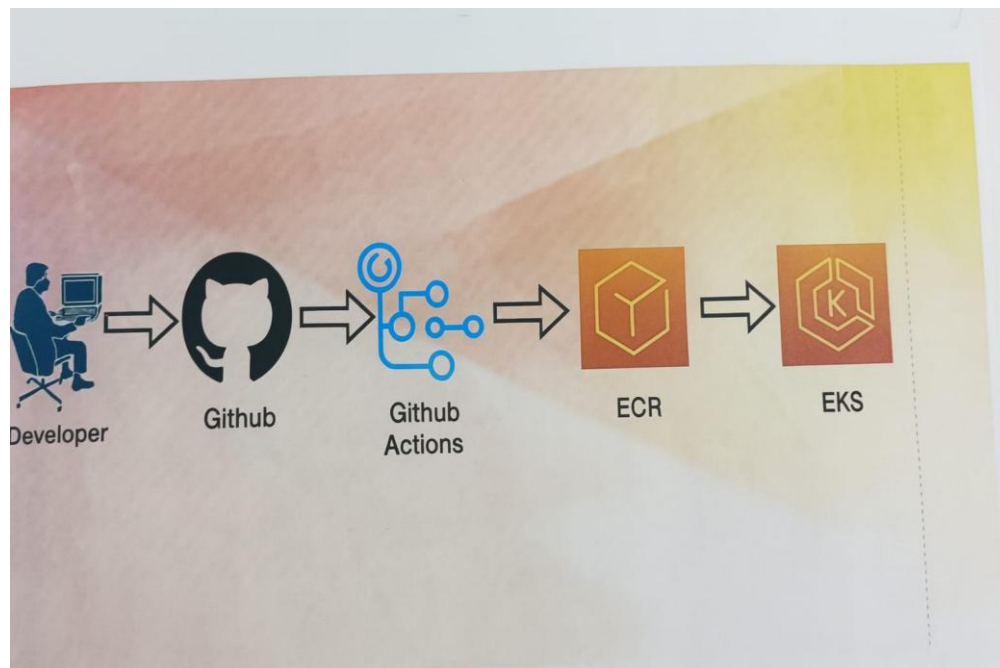


Figure 1: High-Level CI/CD Workflow & Cloud Deployment Architecture

Workflow Stages Breakdown

- **Developer Push:** Developers push code updates and task definitions to the GitHub main repository branch.
- **GitHub Actions Pipeline:** Automatically triggers to set up JDK 17, build Maven artifacts, configure AWS programmable credentials, and build the container image.
- **Amazon ECR:** Stores versioned and immutable Docker container images tagged with unique Git commit SHAs.
- **Amazon ECS Deployment:** Renders updated task definitions and updates the ECS cluster service with zero downtime.

2. Cloud Infrastructure & Network Setup

The cloud infrastructure is provisioned inside an isolated AWS Virtual Private Cloud (VPC) featuring public and private subnets, security groups, and automated NAT routing for secure container egress.

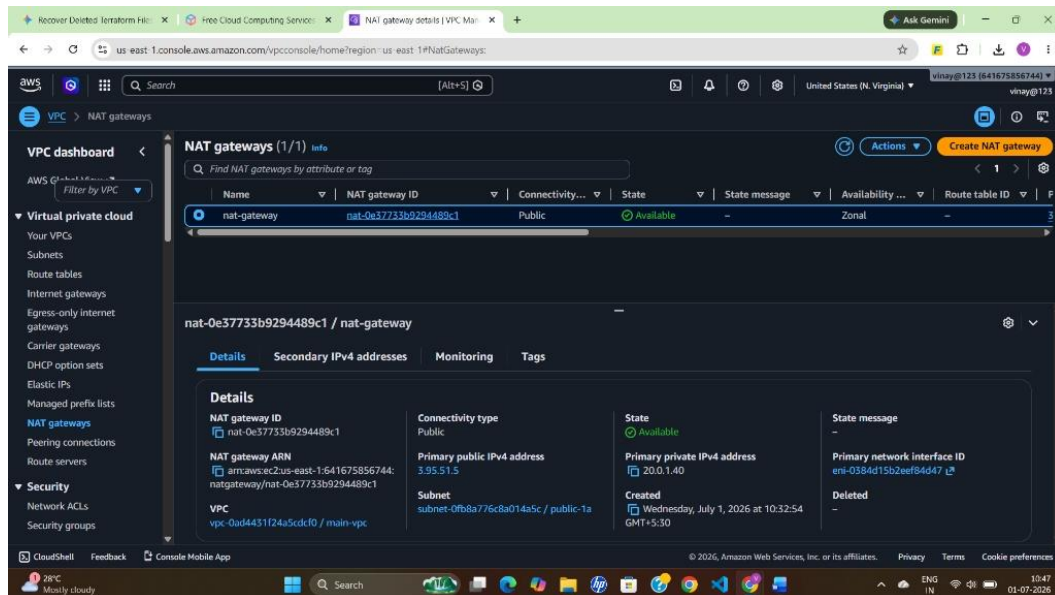


Figure 2: AWS VPC Dashboard — Public NAT Gateway Configuration (nat-0e37733b9294489c1)

- **NAT Gateway:** Configured in public-1a subnet (Elastic IP: 3.95.51.5) to provide secure internet access for containerized backend tasks.
- **EC2 Build & Test Node:** Instance (docker-p / 44.201.114.128) used for local CLI testing, Docker builds, and Amazon ECR registry authentication verification.

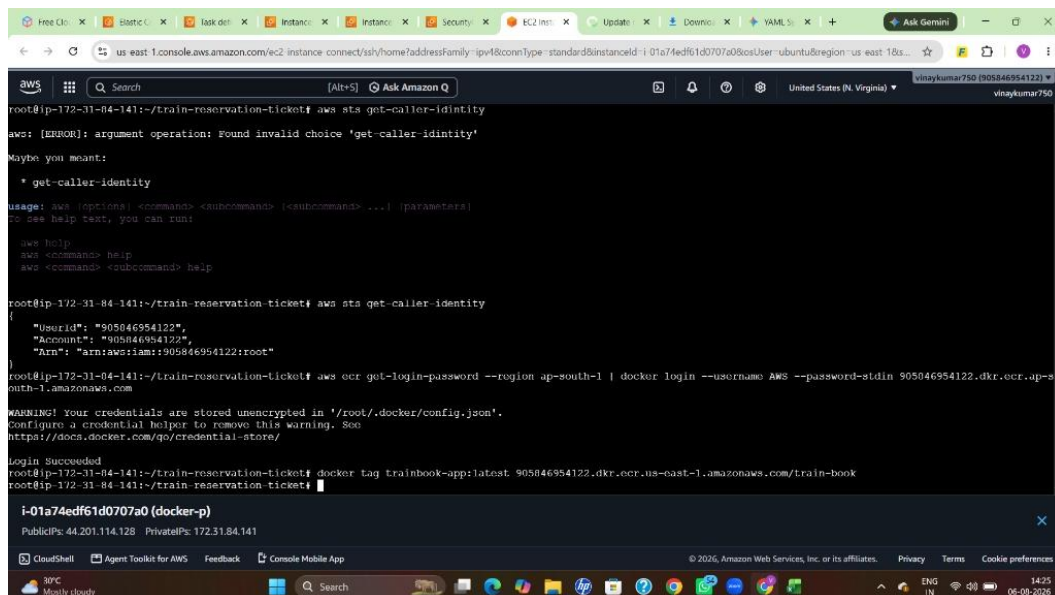


Figure 3: EC2 Instance Terminal — AWS STS Verification, ECR Login & Docker Tagging

3. GitHub Actions Secrets & Pipeline Security

In alignment with security best practices, no credentials or environment-specific values are hardcoded in the codebase. All sensitive variables are securely stored in GitHub Actions Secrets:

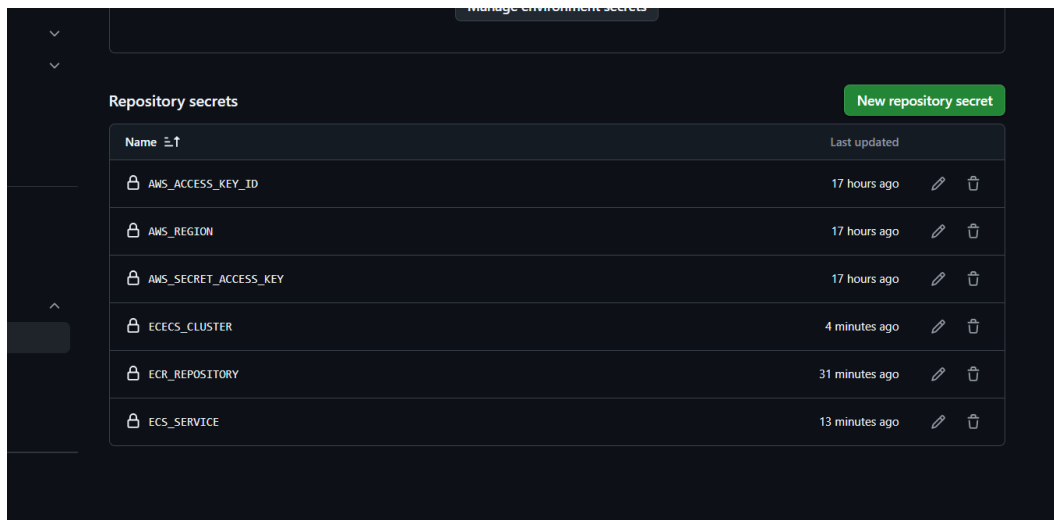


Figure 4: GitHub Repository Secrets Configuration

- **AWS_ACCESS_KEY_ID & AWS_SECRET_ACCESS_KEY:** IAM user credentials for programmatic AWS deployment.
- **AWS_REGION:** Target AWS Region (us-east-1).
- **ECR_REPOSITORY:** Amazon ECR Repository name (train-book).
- **ECS_CLUSTER & ECS_SERVICE:** Target Elastic Container Service cluster (trainbook-cluster) and active service.

4. Pipeline Implementation & GitHub Workflow

The complete GitHub Actions workflow (.github/workflows/main.yml) automates testing, packaging, containerization, and ECS deployment:

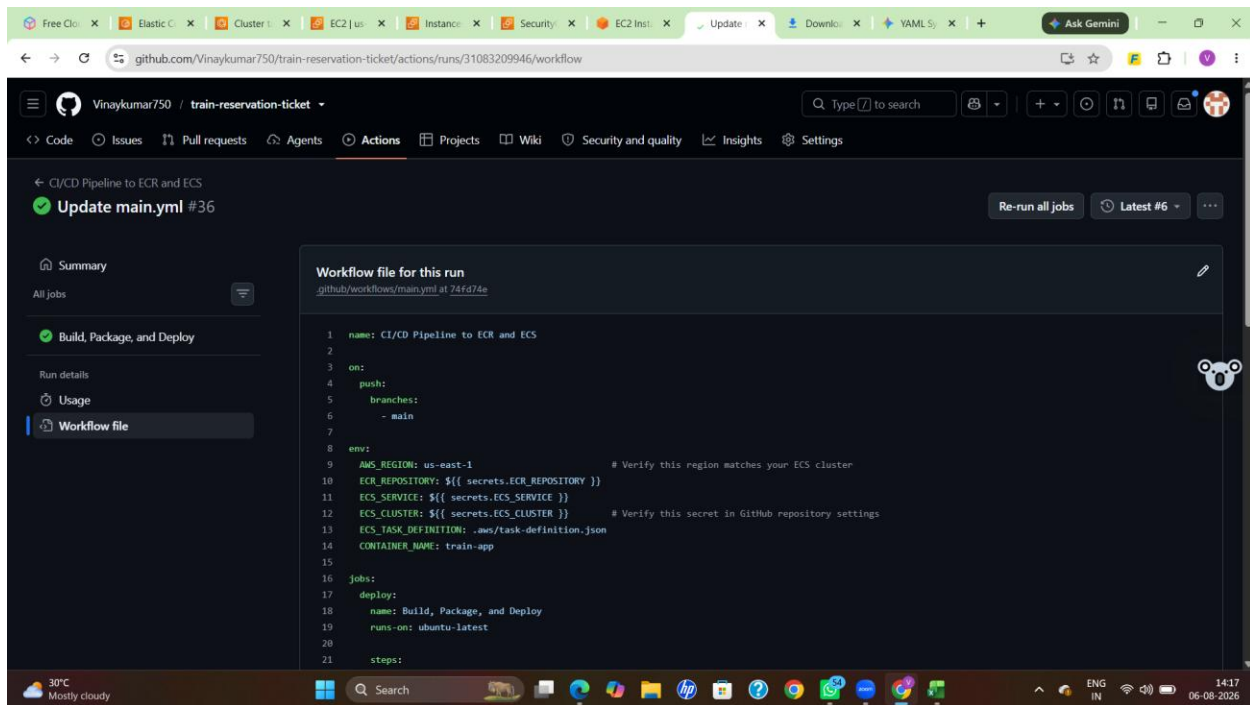


Figure 5: GitHub Actions Workflow YAML Definition (Part 1 - Environment & Triggers)

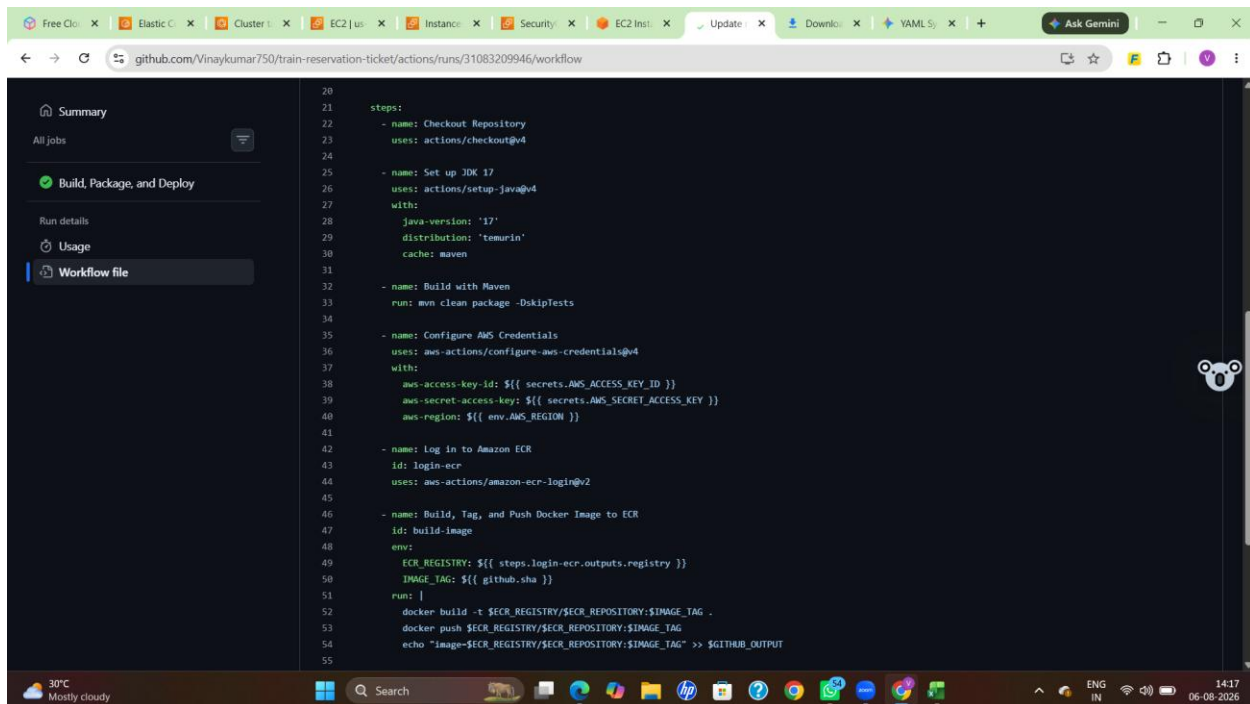


Figure 6: GitHub Actions Workflow YAML Definition (Part 2 - Maven, Docker & ECR Steps)

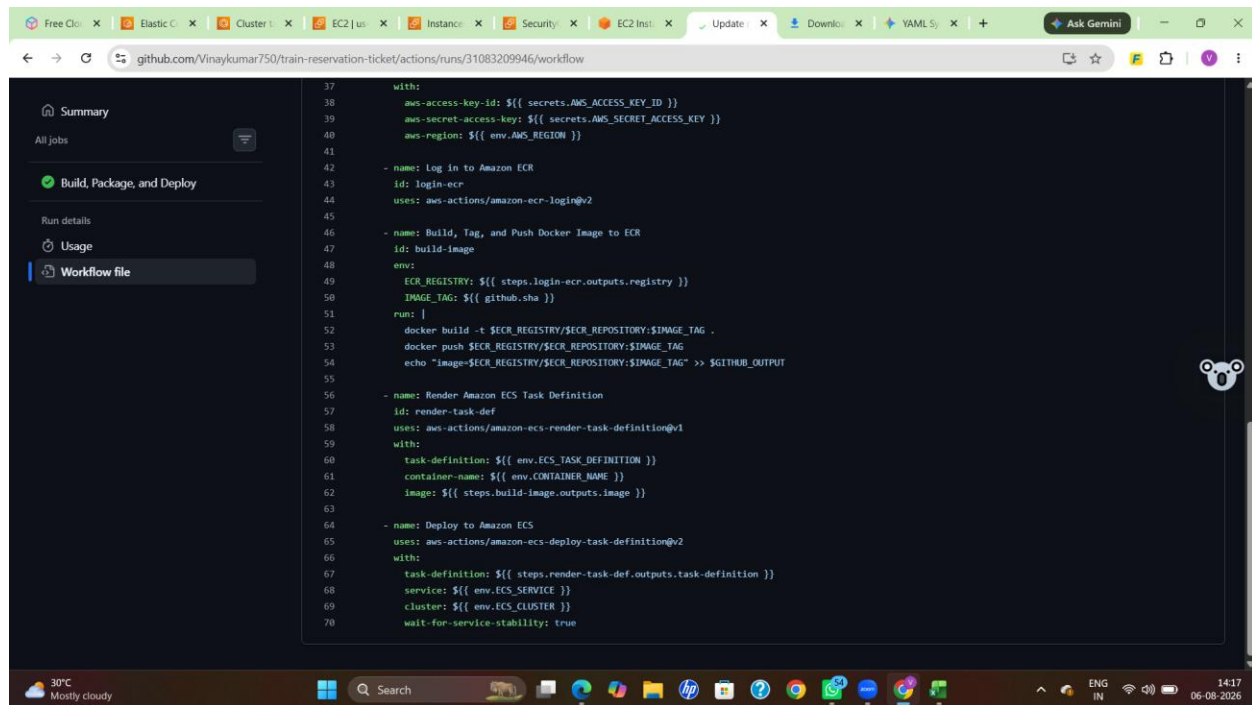


Figure 7: GitHub Actions Workflow YAML Definition (Part 3 - ECS Task Rendering & Deployment)

name: CI/CD Pipeline to ECR and ECS

on:

push:

branches:

- main

env:

AWS_REGION: us-east-1 # Verify this region matches your ECS cluster

ECR_REPOSITORY: \${{ secrets.ECR_REPOSITORY }}

ECS_SERVICE: \${{ secrets.ECS_SERVICE }}

ECS_CLUSTER: \${{ secrets.ECS_CLUSTER }} # Verify this secret in GitHub repository settings

ECS_TASK_DEFINITION: .aws/task-definition.json

CONTAINER_NAME: train-app

jobs:

deploy:

name: Build, Package, and Deploy

runs-on: ubuntu-latest

steps:

- name: Checkout Repository

uses: actions/checkout@v4

- name: Set up JDK 17

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

cache: maven

- name: Build with Maven

run: mvn clean package -DskipTests

- name: Configure AWS Credentials

uses: aws-actions/configure-aws-credentials@v4

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

aws-region: \${{ env.AWS_REGION }}

- name: Log in to Amazon ECR

id: login-ecr

uses: aws-actions/amazon-ecr-login@v2

- name: Build, Tag, and Push Docker Image to ECR

id: build-image

env:

ECR_REGISTRY: \${{ steps.login-ecr.outputs.registry }}

IMAGE_TAG: \${{ github.sha }}

run: |

docker build -t \$ECR_REGISTRY/\$ECR_REPOSITORY:\$IMAGE_TAG .

docker push \$ECR_REGISTRY/\$ECR_REPOSITORY:\$IMAGE_TAG

```
echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT
```

- name: Render Amazon ECS Task Definition

id: render-task-def

uses: aws-actions/amazon-ecs-render-task-definition@v1

with:

task-definition: \${{ env.ECS_TASK_DEFINITION }}

container-name: \${{ env.CONTAINER_NAME }}

image: \${{ steps.build-image.outputs.image }}

- name: Deploy to Amazon ECS

uses: aws-actions/amazon-ecs-deploy-task-definition@v2

with:

task-definition: \${{ steps.render-task-def.outputs.task-definition }}

service: \${{ env.ECS_SERVICE }}

cluster: \${{ env.ECS_CLUSTER }}

wait-for-service-stability: true

5. Deployment Verification & Service Status

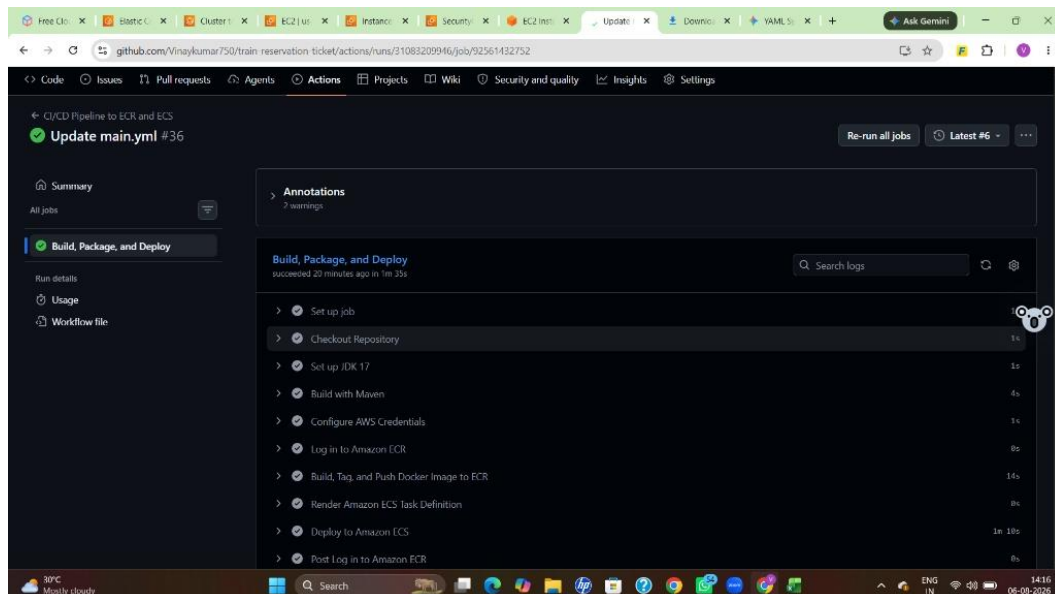


Figure 8: GitHub Actions Workflow Execution — Successful Run (#36)

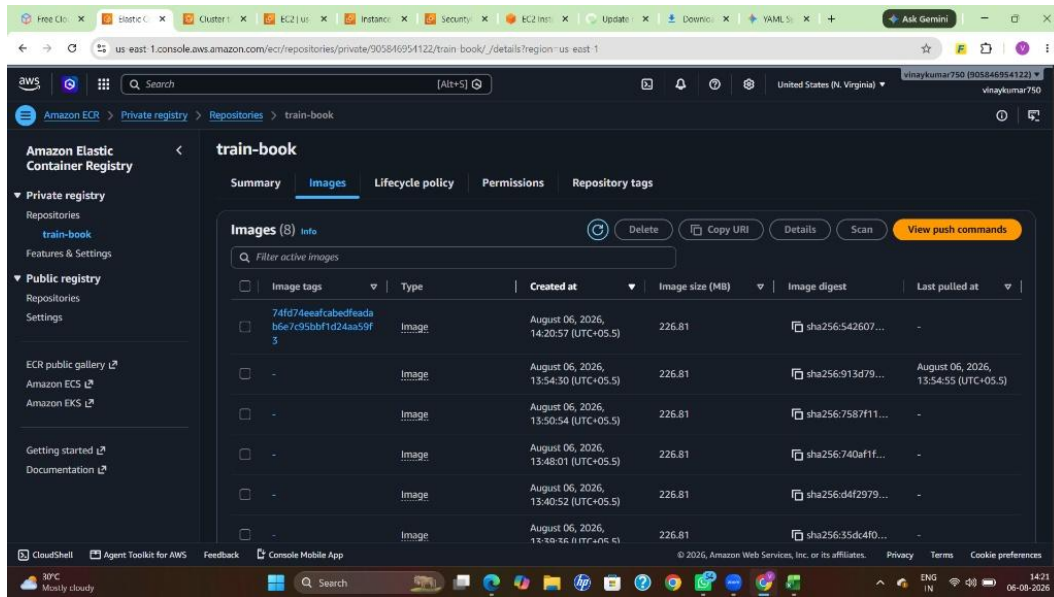


Figure 9: Amazon ECR Private Registry — Versioned Docker Images (train-book)

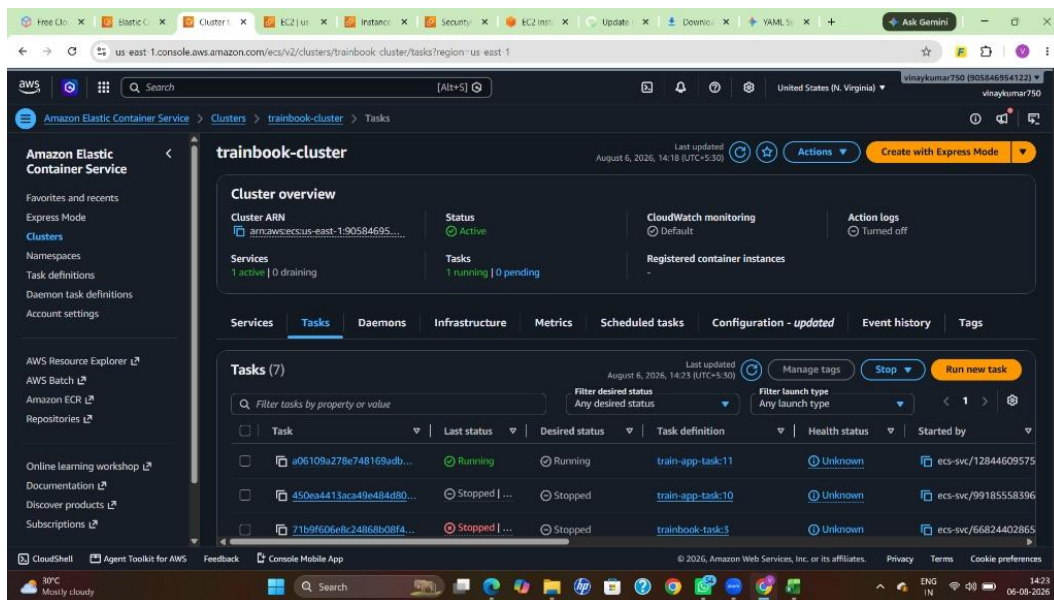


Figure 10: Amazon ECS Cluster Overview — Active Service & Healthy Running Tasks

6. Key Project Highlights & Takeaways

- **Continuous Integration:** Automated code testing and packaging with Maven upon every push to the main branch.
- **Container Management:** Immutable container tags generated per commit SHA and stored securely in Amazon ECR.
- **Zero-Downtime Deployment:** Rolling deployments managed seamlessly by Amazon ECS task definition revisions.
- **Security Compliance:** Granular IAM least-privilege permissions combined with encrypted GitHub Secrets.